5.5 Real Numbers

The branches of mathematics designed to work with real numbers are real important in understanding the real world. Newton had to develop calculus before he could develop the basic laws of motion. The need to integrate or solve systems of equations occurs in every area of science. The first computers were designed as number-crunching machines, and the numbers they were designed to crunch were real numbers.

Working with real numbers on computers is very challenging because floating point arithmetic has limited precision. The most important thing to remember about real numbers is that they are not *real* real numbers:

- Much of mathematics relies on the *continuity* of the reals, the fact that there always exists a number c between a and b if $a < b$. This is not true in real numbers as they are represented in a computer.

- Many algorithms rely on an assumption of *exact* computation. This is not true of real numbers as they are represented in a computer. The associativity of addition guarantees that

$$(a + b) + c = a + (b + c)$$

  Unfortunately, this is not necessarily true in computer arithmetic because of round-off errors.

There are several different types of numbers which we may well want to work with:

- *Integers* — These are the counting numbers, $-\infty, \ldots, -2, -1, 0, 1, 2, \ldots, \infty$. Subsets of the integers include the *natural* numbers (integers starting from 0) and the *positive* integers (those starting from 1), although the notation is not universal. A limiting aspect of integers is that there are gaps between them. An April Fool's Day edition of a newspaper once had a headline announcing, "Scientists Discover New Number Between 6 and 7." This is funny because while there is *always* a rational number between any two rationals x and y ($(x+y)/2$ is a good example), it would indeed be newsworthy if they found an *integer* between 6 and 7.

- *Rational Numbers* — These are the numbers which can be expressed as the ratio of two integers, i.e. c is rational if $c = a/b$ for integers a and b. Every integer can be represented by a rational, namely, $c/1$. The rational numbers are synonymous with fractions, provided we include *improper* fractions a/b where $a > b$.

- *Irrational Numbers* — There are many interesting numbers which are not rational numbers. Examples include $\pi = 3.1415926\ldots$, $\sqrt{2} = 1.41421\ldots$, and $e = 2.71828\ldots$. It can be proven that there does not exist any pair of integers x and y such that x/y equals any of these numbers.

  So how can you represent them on a computer? If you *really* need the values to arbitrary precision, they can be computed using Taylor series expansions. But for all practical purposes it suffices to approximate them using the ten digits or so.

5.5.1 Dealing With Real Numbers

The internal representation of floating point numbers varies from computer to computer, language to language, and compiler to compiler. This makes them a big pain to deal with.

There is an IEEE standard for floating point arithmetic which an increasing number of vendors adhere to, but you must always expect trouble on computations which require very high precision. Floating point numbers are represented in scientific notation, i.e., $a \times 2^c$, with a limited number of bits assigned to both the *mantissa* a and *exponent* c. Operating on two numbers with vastly different exponents often results in overflow or underflow errors, since the mantissa does not have enough bits to accommodate the answer.

Such issues are the source of many difficulties with roundoff errors. The most important problem occurs in testing for equality of real numbers, since there is usually enough garbage in the low-order bits of mantissa to render such tests meaningless. Never test whether a float is equal to zero, or any other float for that matter. Instead, test if it lies within an ϵ value plus or minus of the target.

Many problems will ask you to display an answer to a given number of digits of precision to the right of the decimal point. Here we must distinguish between *rounding* and *truncating*. Truncation is exemplified by the `floor` function, which converts a real number of an integer by chopping off the fractional part. Rounding is used to get a more accurate value for the least significant digit. To round a number X to k decimal digits, use the formula

$$\text{round}(X, k) = \text{floor}(10^k X + (1/2))/10^k$$

Use your language's formatted output function to display only the desired number of digits when so requested.

5.5.2 Fractions

Exact rational numbers x/y are best represented by pairs of integers x, y, where x is the *numerator* and y is the *denominator* of the fraction.

The basic arithmetic operations on rationals $c = x_1/y_1$ and $d = x_2/y_2$ are easy to program:

- *Addition* — We must find a common denominator before adding fractions, so

$$c + d = \frac{x_1 y_2 + x_2 y_1}{y_1 y_2}$$

- *Subtraction* — Same as addition, since $c - d = c + -1 \times d$, so

$$c - d = \frac{x_1 y_2 - x_2 y_1}{y_1 y_2}$$

- *Multiplication* — Since multiplication is repeated addition, it is easily shown that

$$c \times d = \frac{x_1 x_2}{y_1 y_2}$$

- *Division* — To divide fractions you multiply by the *reciprocal* of the denominator, so

$$c/d = \frac{x_1}{y_1} \times \frac{y_2}{x_2} = \frac{x_1 y_2}{x_2 y_1}$$

  But why does this work? Because under this definition, $d(c/d) = c$, which is exactly what we want division to mean.

Blindly implementing these operations leads to a significant danger of overflows. It is important to *reduce* fractions to their simplest representation, i.e., replace 2/4 by 1/2. The secret is to cancel out the *greatest common divisor* of the numerator and the denominator, i.e., the largest integer which divides both of these integers.

Finding the greatest common divisor by trial and error or exhaustive search can be very expensive. However, the Euclidean algorithm for gcd is efficient, very simple to program, and discussed in Section 7.2.1.

5.5.3 Decimals

The decimal representation of real numbers is just a special case of the rational numbers. A decimal number represents the sum of two numbers; the integer part to the left of the decimal point and the fractional part to the right of the decimal. Thus the fractional representation of the first five decimal digits of π is

$$3.1415 = (3/1) + (1415/10000) = 6283/2000$$

The denominator of the fractional part is 10^{i+1} if the rightmost non-zero digit lies i places to the right of the decimal point.

Converting a rational to a decimal number is easy, in principle; just divide the numerator by the denominator. The catch is that many fractions do not have a finite decimal representation. For example, $1/3 = 0.3333333\ldots$, and $1/7 = 0.14285714285714285714\ldots$. Usually a decimal representation with the first ten or so significant digits will suffice, but sometimes we want to know the exact representation, i.e., $1/30 = 0.0\overline{3}$ or $1/7 = 0.\overline{142857}$.

What fraction goes with a given repeating decimal number? We can find it by explicitly simulating the long division. The decimal expansion of fraction 1/7 is obtained by dividing 7 into $1.0000000\ldots$. The next digit of the quotient is obtained by multiplying the remainder by ten, adding the last digit (always zero), and finding how many times the denominator fits into this quantity. Notice that we get into an infinite loop the instant this quantity repeats. Thus the decimal digits between these positions repeats forever.

A simpler method results if we know (or guess) the length of the repeat. Suppose that the simple fraction a/b has a repeat R of length of l. Then $10^l(a/b) - (a/b) = R$, and

hence $a/b = R/(10^l - 1)$. To demonstrate, suppose we want the fraction associated with $a/b = 0.0123123\ldots$. The repeat length is three digits, and $R = 12.3$ by the formula above. Thus $a/b = 12.3/999 = 123/9990$.

5.6 Algebra

In its full glory, algebra is the study of groups and rings. High-school algebra is basically limited to the study of equations, defined over the operators addition and multiplication. The most important class of formulae are the *polynomials*, such that $P(x) = c_0 + c_1 x + c_2 x^2 + \ldots$, where x is the variable and c_i is the coefficient of the ith term x^i. The degree of a polynomial is the largest i such that c_i is non-zero.

5.6.1 Manipulating Polynomials

The most natural representation for an nth-degree univariate (one variable) polynomial is as an array of $n + 1$ coefficients c_0 through c_n. Such a representation makes short work of the basic arithmetic operations on polynomials:

- *Evaluation* — Computing $P(x)$ for some given x can easily be done by brute force, namely, computing each term $c_i x^n$ independently and adding them together. The trouble is that this will cost $O(n^2)$ multiplications where $O(n)$ suffice. The secret is to note that $x^i = x^{i-1}x$, so if we compute the terms from smallest degree to highest degree we can keep track of the current power of x, and get away with two multiplications per term ($x^{i-1} \times x$, and then $c_i \times x^i$).

  Alternately, one can employ *Horner's rule*, an even slicker way to do the same job:

$$a_n x^n + a_{n-1} x^{n-1} + \ldots + a_0 = ((a_n x + a_{n-1})x + \ldots)x + a_0$$

- *Addition/Subtraction* — Adding and subtracting polynomials is even easier than the same operations on long integers, since there is no borrowing or carrying. Simply add or subtract the coefficients of the ith terms for all i from zero to the maximum degree.

- *Multiplication* — The product of polynomials $P(x)$ and $Q(x)$ is the sum of the product of every pair of terms, where each term comes from a different polynomial:

$$P(x) \times Q(x) = \sum_{i=0}^{degree(P)} \sum_{j=0}^{degree(Q)} (c_i c_j) x^{i+j}$$

  Such an all-against-all operation is called a *convolution*. Other convolutions in this book include integer multiplication (all digits against all digits) and string matching (all possible positions of the pattern string against all possible text positions). There is an amazing algorithm (the fast Fourier transform, or FFT) which computes convolutions in $O(n \log n)$ time instead of $O(n^2)$, but it is well

beyond the scope of this book. Still, it is nice to recognize when you are doing a convolution so you know that such tools exist.

- *Division* — Dividing polynomials is a tricky business, since the polynomials are not closed under division. Note that $1/x$ may or may not be thought of as a polynomial, since it is x^{-1}, but $2x/(x^2+1)$ certainly isn't. It is a *rational function*.

Sometimes polynomials are *sparse*, meaning that for many coefficients $c_i = 0$. Sufficiently sparse polynomials should be represented as linked lists of coefficient/degree pairs. *Multivariate polynomials* are defined over more than one variable. The bivariate polynomial $f(x, y)$ can be represented by a matrix C of coefficients, such that $C[i][j]$ is the coefficient of $x^i y^j$.

5.6.2 Root Finding

Given a polynomial $P(x)$ and a target number t, the problem of *root finding* is identifying any or all x such that $P(x) = t$.

If $P(x)$ is a first-degree polynomial, the root is simply $x = (t - a_0)/a_1$, where a_i is the coefficient of x_i in $P(x)$. If $P(x)$ is a second-degree polynomial, then the *quadratic equation* applies:

$$x = \frac{-a_1 \pm \sqrt{{a_1}^2 - 4a_2(a_0 - t)}}{2a_2}$$

There are more complicated formulae for solving third- and fourth-degree polynomials before the good times end. No closed form exists for the roots of fifth-degree (quintic) or higher-degree equations.

Beyond quadratic equations, numerical methods are typically used. Any text on numerical analysis will describe a variety of root-finding algorithms, including Newton's method and Newton-Raphson, as well as many potential traps such as numerical stability. But the basic idea is that of binary search. Suppose a function $f(x)$ is *monotonically increasing* between l and u, meaning that $f(i) \leq f(j)$ for all $l \leq i \leq j \leq u$. Now suppose we want to find the x such that $f(x) = t$. We can compare $f((l + u)/2)$ with t. If $t < f((l + u)/2)$, then the root lies between l and $(l + u)/2$; if not, it lies between $(l+u)/2$ and u. We can keep recurring until the window is narrow enough for our taste.

This method can be used to compute square roots because this is equivalent to solving $x^2 = t$ between 1 and t for all $t \geq 1$. However, a simpler method to find the ith root of t uses exponential functions and logarithms to compute $t^{1/i}$.

5.7 Logarithms

You have probably noticed the `log` and `exp` buttons on your calculator, but quite likely have never used them. You may even have forgotten why they are there. A *logarithm* is simply an inverse exponential function. Saying $b^x = y$ is equivalent to saying that $x = \log_b y$.

The b term is known as the *base* of the logarithm. Two bases are of particular importance for mathematical and historical reasons. The *natural log*, usually denoted $\ln x$, is a base $e = 2.71828\ldots$ logarithm. The inverse of $\ln x$ is the exponential function $\exp x = e^x$. Thus by composing these functions we get

$$\exp(\ln x) = x$$

Less common today is the base-10 or *common logarithm*, usually denoted $\log x$. Common logarithms were particularly important in the days before pocket calculators.[1] Logarithms provided the easiest way to multiply big numbers by hand, either implicitly using a slide rule or explicitly by using a book of logarithms.

Logarithms are still useful for multiplication, particularly for exponentiation. Recall that $\log_a xy = \log_a x + \log_a y$; i.e., the log of a product is the sum of the logs. A direct consequence of this is that

$$\log_a n^b = b \cdot \log_a n$$

So how can we compute a^b for any a and b using the $\exp(x)$ and $\ln(x)$ functions? We know

$$a^b = \exp(\ln(a^b)) = \exp(b \ln a)$$

so the problem is reduced to one multiplication plus one call of each of these functions.

We can use this method to compute square roots since $\sqrt{x} = x^{1/2}$, and for any other fractional power as well. Such applications are one reason why the mathematics library of any reasonable programming language includes the ln and exp functions. Be aware that these are complicated numerical functions (computed using Taylor-series expansions) which have inherent computational uncertainty, so do not expect that $\exp(0.5 \ln 4)$ will give you exactly 2.

The other important fact to remember about logarithms is that it is easy to convert the logarithm from one base to another, as a consequence of the following formula:

$$\log_a b = \frac{\log_c b}{\log_c a}$$

Thus changing the base of $\log b$ from base-a to base-c simply involves dividing by $\log_c a$. Thus it is easy to write a common log function from a natural log function, and vice versa.

5.8 Real Mathematical Libraries

Math Libraries in C/C++

The standard C/C++ math library has several useful functions for working with real numbers:

[1] The authors of this book are old enough to remember this pre-1972 era.

```
#include <math.h>          /* include the math library */

double floor(double x);    /* chop off fractional part of x */
double ceil (double x);    /* raise x to next largest integer */
double fabs(double x);     /* compute the absolute value of x */

double sqrt(double x);     /* compute square roots */
double exp(double x);      /* compute e^x */
double log(double x);      /* compute the base-e logarithm */
double log10(double x);    /* compute the base-10 logarithm */
double pow(double x, double y);   /* compute x^y */
```

Math Libraries in Java

The java class *java.lang.Math* has all of these functions and a few more, most obviously a `round` function to take a real to the nearest integer.